Introduzione a XML

ML, acronimo di eXtensible Markup Language, è un linguaggio di markup pensato per descrivere dati in modo strutturato, gerarchico e indipendente dalla piattaforma.

Tecnicamente non nasce per “mostrare” contenuti come HTML, ma per rappresentare informazioni in una forma leggibile, trasportabile e interpretabile da software diversi.

La sua idea centrale è separare il contenuto dalla logica applicativa: invece di dire come qualcosa deve apparire, XML descrive cosa rappresenta quel dato.

Per questo viene usato come formato di interscambio, configurazione e memorizzazione strutturata.

La parola estensibile è fondamentale. In XML i tag non sono predefiniti come in HTML, ma possono essere creati in base al dominio applicativo.

Questo significa che un’azienda, un software o uno standard possono definire tag come <cliente>, <fattura>, <prodotto>, <corso> o <studente> per modellare i propri dati.

In questo senso XML è un metalinguaggio: non impone un vocabolario fisso, ma fornisce le regole per costruire linguaggi di marcatura personalizzati.

Utilizzi principali

- Scambio dati tra sistemi (es. web services SOAP)
- Configurazioni software
- Persistenza strutturata di dati
- Formati standard (es. RSS, SVG, XHTML

Struttura tecnica

XML utilizza una struttura ad albero (tree structure) composta da elementi annidati:

- **Elemento (tag): definisce una porzione di dati**
- **Attributo: fornisce metadati sull’elemento**
- **Testo: contenuto interno dell’elemento**

```
<persona id="1">
    <nome>Mario</nome>
    <cognome>Rossi</cognome>
</persona>
```

Dal punto di vista strutturale, un documento XML è organizzato come un albero.

Esiste un elemento radice che contiene tutti gli altri elementi, e ciascun elemento può avere sottoelementi, attributi e contenuto testuale.

Questa struttura gerarchica lo rende adatto a rappresentare relazioni annidate, come ad esempio un ordine che contiene cliente, prodotti, quantità e totale.

Ogni nodo dell’albero ha un significato ben preciso e può essere elaborato da parser o applicazioni che leggono il documento.

Un esempio semplice può essere questo:

```
1. <studente matricola="1024">
2.     <nome>Marco</nome>
3.     <cognome>Bianchi</cognome>
4.     <corso>Informatica</corso>
5. </studente>
```

In questo caso studente è l'elemento principale, matricola è un attributo, mentre nome, cognome e corso sono elementi figli.

Il documento descrive i dati, non la loro presentazione grafica. Sarà poi il programma che lo legge a decidere come usarli.

Caratteristiche principali

- **Estensibile: i tag non sono predefiniti (a differenza di HTML), ma definiti dall'utente**
- **Gerarchico: struttura ad albero con nodi padre/figlio**
- **Autodescrittivo: i dati includono il loro significato tramite i tag**
- **Portabile: indipendente da piattaforma e linguaggio**
- **Testuale: formato leggibile e facilmente parsabile**

Uno degli aspetti più importanti di XML è la sua rigidità sintattica.

Questa rigidità serve a garantire che i dati siano chiari, coerenti e interpretabili senza ambiguità. I tag devono essere sempre chiusi correttamente, devono essere annidati in modo ordinato, gli attributi devono avere valori racchiusi tra virgolette e deve esistere un solo elemento radice.

Inoltre XML distingue maiuscole e minuscole, quindi <Nome> e <nome> sono considerati elementi diversi.

Tutte queste regole fanno sì che un documento XML possa essere letto in modo affidabile da parser automatici.

Regole sintattiche fondamentali

- Un documento deve avere un solo elemento radice
- Tutti i tag devono essere correttamente chiusi
- I tag sono case-sensitive
- Gli attributi devono essere racchiusi tra virgolette
- La struttura deve essere ben formata (well-formed)

Validazione

Un documento XML può essere:

- **Well-formed: rispetta le regole sintattiche**
- **Valid: rispetta anche una struttura definita tramite:**
  - **DTD (Document Type Definition)**
  - **XSD (XML Schema Definition)**

Qui entra in gioco il concetto di validazione, che è uno dei punti più tecnici e importanti.

Prima di tutto bisogna distinguere tra documento well-formed e documento valid.

Un documento well-formed è un documento che rispetta le regole sintattiche di XML.

In pratica è scritto bene dal punto di vista formale. Se invece un tag viene chiuso male, o c’è un elemento annidato in modo scorretto, il documento non è neanche considerato XML corretto.

Esempio non corretto:

```
1. <persona>
2.     <nome>Luca</nome>
3.     <cognome>Verdi
4. </persona>
```

Qui cognome non viene chiuso, quindi il documento non è ben formato; un documento valid, invece, oltre a essere ben formato, rispetta anche una struttura prevista da uno schema esterno.

In altre parole non basta che il file sia sintatticamente corretto: deve anche contenere solo gli elementi ammessi, nell'ordine previsto, con i tipi di dato corretti e con eventuali vincoli richiesti.

I due principali strumenti di validazione storicamente usati con XML sono DTD e XSD. La DTD, cioè Document Type Definition, è il sistema più vecchio.

Permette di definire quali elementi sono ammessi, come possono essere annidati e quali attributi possono comparire.

È utile per definire la struttura generale, ma è limitata perché non gestisce bene i tipi di dato moderni.

Per esempio non può esprimere in modo preciso che un certo valore debba essere un intero, una data o una stringa con un formato particolare.

```
1. <!DOCTYPE studente [
2.     <!ELEMENT studente (nome, cognome, corso)>
3.     <!ATTLIST studente matricola CDATA #REQUIRED>
4.     <!ELEMENT nome (#PCDATA)>
5.     <!ELEMENT cognome (#PCDATA)>
6.     <!ELEMENT corso (#PCDATA)>
7. ]>
```

Questa definizione dice che studente deve contenere nell'ordine nome, cognome e corso, e che deve avere un attributo matricola obbligatorio.

Più potente è invece XSD, cioè XML Schema Definition. XSD è anch'esso scritto in XML e permette di definire in modo molto più rigoroso la struttura del documento.

Con XSD si possono specificare tipi di dato come stringhe, interi, date, booleani, lunghezze minime e massime, pattern, cardinalità minima e massima degli elementi e altro ancora. In un contesto tecnico o aziendale, XSD è molto più usato rispetto a DTD proprio perché offre controlli più forti.

Un esempio semplificato di XSD è questo:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="studente">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="nome" type="xs:string"/>
        <xs:element name="cognome" type="xs:string"/>
        <xs:element name="corso" type="xs:string"/>
      </xs:sequence>
      <xs:attribute name="matricola" type="xs:int" use="required"/>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Qui viene imposto che matricola sia un intero obbligatorio, e che gli elementi figli compaiano in una sequenza precisa.

Se nel documento XML il valore di matricola non fosse numerico, oppure mancasse uno degli elementi richiesti, la validazione fallirebbe.

Il vantaggio della validazione è enorme nei sistemi software. Quando più applicazioni si scambiano dati, non basta ricevere un file formalmente corretto: bisogna essere sicuri che contenga davvero ciò che ci si aspetta.

Parsing

Per leggere XML si utilizzano parser:

- **DOM (Document Object Model) → carica tutto in memoria (navigazione ad albero)**
- **SAX (Simple API for XML) → parsing sequenziale, più leggero**

Dal punto di vista operativo, la validazione viene fatta da parser XML con supporto a DTD o XSD.

Un parser legge il documento, ne verifica la correttezza sintattica e, se richiesto, confronta il contenuto con lo schema associato.

Se trova errori, restituisce messaggi che indicano dove la struttura non rispetta le regole previste.

Questo è molto utile sia in fase di sviluppo sia in ambienti produttivi dove il controllo dei dati è cruciale.

Buon Davante a tutti